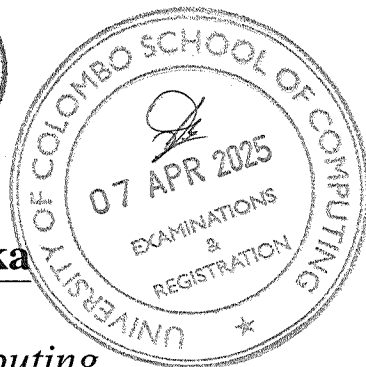


UCSC

University of Colombo, Sri Lanka

University of Colombo School of Computing

Bachelor of Science in Computer Science

First Year Examination — Semester II— UCSC AY21 [held in March/April 2025]

SCS1312 — Operating System Concepts

(2 Hours)

Answer All Questions

Number of Pages = 12

Number of Questions = 4

To be completed by the candidate

Index Number

--	--	--	--	--	--	--	--

140

Important Instructions to candidates:

- Students should answer in the medium of English language only using the space provided in this question paper.
- Note that questions appear on both sides of the paper. If a page or a part of this question paper is not printed, please inform the supervisor immediately.
- Write your index number **CLEARLY** on each and every page of this Question paper.
- This paper consists of **4** questions in **12** pages (including the Cover Page).
- Answer **ALL** questions.
- Programmable Calculators and any electronic device capable of storing and retrieving text including electronic dictionaries, smart watches and mobile phones are not allowed.
- Non-Programmable calculators are allowed
- Do not tear off any part of this answer book. Under no circumstances may this book, used or unused, be removed from the Examination Hall by a candidate

To be completed by the examiners

1	
2	
3	
4	
Total	

Index Number

--	--	--	--	--	--	--	--

1. (a). A code segment (Python) of the virtual machine used for the SCS1312 assignments is given below.

```
....  
  
def add(opr):  
    reg['acc']=reg[opr[0]]+reg[opr[1]];  
    reg['pc']=reg['pc']+1;  
  
def call(opr):  
    reg['sp']=reg['sp']+1;  
    memory[reg['sp']]=reg['pc']+1;  
    reg['pc']=X;  
  
def ret(opr):  
    reg['pc']=memory[reg['sp']];  
    reg['sp']=reg['sp']-1;  
  
def push(opr):  
    reg['sp']=reg['sp']+1;  
    Y=reg[opr[0]];  
    reg['pc']=reg['pc']+1;  
  
def pop(opr):  
    reg[opr[0]]=memory[reg['sp']];  
    reg['sp']=Z;  
    reg['pc']=reg['pc']+1;  
  
def jnz(opr):  
    if reg[opr[1]]!=0:  
        Q;  
    else:  
        reg['pc']=reg['pc']+1;  
  
....
```

- i. What is X?

[3 marks]

--

- ii. What is Y?

[3 marks]

--

Index Number

--	--	--	--	--	--	--	--

iii. What is Z?

[3 marks]

--

iv. What is Q?

[3 marks]

--

(b). The questions 1(b)i and 1(b)iv refers to a system that uses a single level page table and a page size of p .

The logical address a_1 is mapped to a physical address in the frame f_1 . Another page is mapped to the frame f_2 . Only two pages are mapped in the page table.

Note: You can use Python mathematical operators to express your answers.

i. What is the physical address of a_1 ?

[3 marks]

--

ii. What is the page number that contains the logical address a_1 ?

[3 marks]

--

iii. The logical address a_2 has the following property.

$$(a_2 - a_1) // p = 0$$

What is the physical address of a_2 ?

[4 marks]

--

iv. If the length of an address in this system is $\log(n) + \log(p)$, how many entries are there in the page table?

[3 marks]

--

Index Number

--	--	--	--	--	--	--	--

2. (a). i. What is the output of the following C programme?

```
int main()
{
    int x;
    x=7;
    fork();
    x++;
    fork();
    x++;
    printf("%d\n", x);
}
```

[4 marks]

--

- ii. What is the output of the following C programme? Assume that all fork() calls completed successfully.

```
int main()
{
    int x=10;
    if (fork())
        if (!(x=fork()))
            printf("%d\n", x);
}
```

[4 marks]

--

- iii. What is the output of the following C programme? Assume that all fork() calls completed successfully.

```
int main()
{
    int x=1;
    while(fork() && x)
    {
        printf("%d\n", x);
        x--;
    }
}
```

[5 marks]

--

Index Number

--	--	--	--	--	--	--	--

- (b). The structure of the Bounded-waiting Mutual Exclusion with `test_and_set` is given below. **X, Y, Z, Q** are place holders for the actual code segments.

```
do {  
    waiting[i] = true;  
    key = true;  
    while (X)  
        key = test_and_set(&lock);  
    waiting[i] = false;  
    /* critical section */  
    j = (i + 1) % n;  
    while ((j != i) && !waiting[j])  
        Y;  
    if (j == i)  
        lock = Z;  
    else  
        waiting[j] = Q;  
    /* remainder section */  
} while (true);
```

i. What is **X**?

[3 marks]

--

ii. What is **Y**?

[3 marks]

--

iii. What is **Z**?

[3 marks]

--

iv. What is **Q**?

[3 marks]

--

--	--	--	--	--	--	--	--

- | Process | Burst Time (ms) |
|---------|-----------------|
| P1 | 20 |
| P2 | 2 |
| P3 | 3 |
| P4 | 1 |

- [4 marks]**

- [5 marks]**

[5 marks]

- [6 marks]**

--

--	--	--	--	--	--	--	--

- | Process | Burst Time (ms) |
|---------|-----------------|
| P1 | 10 |
| P2 | 5 |
| P3 | 7 |
| P4 | 12 |

- [4 marks]**

[REDACTED]

- CPU Time
- Starvation
- Job finishing order

Index Number

--	--	--	--	--	--	--	--

- iii. If the time quantum is increased to 6 ms, how does it impact the average waiting time? Would this still maintain fairness? Explain your answer considering following factors:
- Context switches
 - Order of process executions and consumed cycles for completion

[3 marks]

--

Index Number

--	--	--	--	--	--	--	--

4. (a). Consider questions (a)-i and (a)-ii based on virtualization.

i. A company runs multiple applications requiring different operating systems on the same hardware. The system has:

- Hardware: x86-64 CPU
- Host OS: Linux
- Guest OS: Windows and FreeBSD
- Hypervisor: Type-1 (bare-metal)

What is the most suitable virtualization technique that can be used for the above scenario?

[2 marks]

--

ii. A cloud provider wants to offer ARM-based virtual machines on a x86-based server infrastructure. The system has:

- Host Machine: x86-64 architecture.
- Guest OS: Linux for ARM.
- Virtualization Requirement: The ARM guest OS must run on x86 hardware without modification.

What is the most suitable virtualization technique that can be used for the above scenario?

[2 marks]

--

(b). A system has a 32-bit virtual address space and uses a page size of 8 KB. The physical memory is 512 MB, and each page table entry takes 4 bytes. The system supports swapping, meaning that pages not currently in use may be moved to disk storage.

i. Determine how many pages are needed to cover the entire virtual address space.

[2 marks]

--

Index Number

--	--	--	--	--	--	--	--

- ii. Find out how much space the page table requires in memory if all entries are stored in a single-level page table.

[2 marks]

--

- iii. If 10% of the pages are swapped out to disk, calculate the amount of disk space required for storing these swapped-out pages.

[4 marks]

--

Index Number

--	--	--	--	--	--	--	--

iv. A hierarchical page table uses multiple levels to manage memory more efficiently. Let's assume a two-level paging system:

- The outer page table holds pointers to inner page tables.
- Each inner page table maps a portion of the address space.

Compute memory usage in hierarchical (multilevel with 2 levels eg: outer and inner levels) page table . Clearly list all assumptions made for the calculation.

[4 marks]

--

(c). A file system uses indexed allocation to store files. Consider a file of 50 KB that needs to be stored on a disk with a block size of 4 KB.

i. How many blocks are needed to store the file?

[2 marks]

--

ii. If each index block can hold 256 block pointers (each pointer is 4 bytes), Briefly justify whether a single index block is enough to store the above file mentioned in c (i).

[2 marks]

--

Index Number

--	--	--	--	--	--	--	--

- (d). A system has 4 page frames, and the following page reference string is given:
2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2

Using the Clock Page Replacement Algorithm (unmodified version) , determine the following :

- i. How many bits are used in the system to track memory changes ? Briefly define them.

[2 marks]

--

- ii. How many page-faults are triggered for the above referenced string using the *Clock Page Replacement Algorithm*?

[1 marks]

--

- iii. What is the final contents of the page frame after processing the reference string (provide content of all 4 frames) ?

[2 marks]

--
